

实验四 二叉树的综合应用

一、实验目的

掌握二叉树的遍历算法，熟练使用遍历算法进行问题的求解。

二、实验内容

对于给定的字符串“ABF#C#G##DE##H###”，用递归的方法实现以下算法：

- (1) 以二叉链表表示二叉树，建立一棵二叉树；
- (2) 输出二叉树的中序遍历结果；
- (3) 输出结点“C”的左右孩子的值；
- (4) 计算二叉树的深度；
- (5) 统计二叉树的叶子结点个数；
- (6) 统计二叉树的度为

```
int main()//测试用例 ABF#C#G##DE##H###
{
    printf("👉请输入入树元素: \n");
    BiTree T;
    CreateBiTree(&T);
    printf("前序遍历: ");
    PreOrderTraverse (T);
    printf("\n");
    printf("中序遍历: ");
    InOrderTraverse(T);
    printf("\n");
    printf("后续遍历: ");
    PostOrderTraverse(T);
    printf("\n");
    printf("叶子结点: ");
    LeafOfTree(T);
    printf("\n");
    printf("\n叶子节点个数为: %d", Get_Leaf_Num(T));
    printf("\n");
    printf("\n二叉树的高度(深度)为: %d", Get_Height(T));
    return 0;
}
```

;

```

4     typedef struct BiTNode
5     {
6         char data;
7         struct BiTNode *lchild, *rchild;
8     }BiTNode,*BiTree;
9
10    void CreateBiTree(BiTree *T) //先序创建二叉树
11    {
12        char ch;
13        scanf("%c", &ch);
14        if(ch == '#')
15            *T = NULL;
16        else
17        {
18            *T = (BiTree )malloc(sizeof(BiTNode));
19            if(!*T)
20                exit(-1);
21            (*T)->data = ch;
22            CreateBiTree(&(*T)->lchild);    //递归创建左、右孩子
23            CreateBiTree(&(*T)->rchild);
24        }
25    }
26
27    void PreOrderTraverse(BiTree T)//二叉树的先序遍历
28    {
29        if(T == NULL)
30            return ;
31        printf("%c ",T->data);
32        PreOrderTraverse(T->lchild);
33        PreOrderTraverse(T->rchild);
34    }
35
36    void InOrderTraverse(BiTree T)//二叉树的中序遍历
37    {
38        if(T == NULL)
39            return ;
40        InOrderTraverse(T->lchild);
41        printf("%c ",T->data);
42        InOrderTraverse(T->rchild);
43    }
44
45    void PostOrderTraverse(BiTree T)//后序遍历
46    {
47        if(T==NULL)
48            return;
49        PostOrderTraverse(T->lchild);
50        PostOrderTraverse(T->rchild);
51        printf("%c ",T->data);
52    }

```

```

55 // 打印树的叶子节点函数定义
56 void LeafOfTree(BiTree T) {
57     if (T == NULL)
58         return ;
59
60     else {
61         if (T->lchild == NULL && T->rchild == NULL)
62             putchar(T->data);
63         else {
64             LeafOfTree(T->lchild);
65             LeafOfTree(T->rchild);
66         }
67     }
68
69 }
70
71 // 获取树的叶子节点个数函数定义
72 int Get_Leaf_Num(BiTree T) {
73     if (T == NULL)
74         return 0;
75     if (T->lchild == NULL && T->rchild == NULL)
76         return 1;
77     //递归整个树的叶子节点个数 = 左子树叶子节点的个数 + 右子树叶子节点的个数
78     return Get_Leaf_Num(T->lchild) + Get_Leaf_Num(T->rchild);
79 }
80 // 获取树高的函数定义
81 int Get_Height(BiTree T) {
82     int Height = 0;
83     if (T == NULL)
84         return 0;

```

😊 请输入入树元素：

ABF#C#G##DE##H###

前序遍历： A B F C G D E H

中序遍历： F C G B E D H A

后续遍历： G C F E H D B A

叶子结点： GEH

叶子节点个数为： 3

二叉树的高度（深度）为： 5%